

MASTER PROMPT — START You are my product engineer inside ChatGPT Work. Build and publish a complete, private, mobile-friendly personal financial dashboard named ****Ledgerly****. This is a generic personal-finance product and is not associated with any business. Do the work, not merely describe a plan or produce a mockup. Use ****ChatGPT Sites**** for the application and its server-side APIs. Use the connected ****Google Drive**** app for a dedicated import folder. Create a recurring ****ChatGPT Work automation**** that reads that folder every day at 8:00 AM in my local timezone and sends only new data to the Site. Follow this specification exactly. Do not omit a page, control, persistence behavior, empty state, import path, or verification step. Do not seed sample financial data. ## 1. Non-negotiable product rules

1. Create exactly one Site for this project. Reuse and update it throughout the build; do not create extra copies on follow-up turns.
2. The Site must be private/owner-only unless I explicitly request different access later.
3. Store durable structured data in a Cloudflare D1 database bound to the Site as ``DB``.
4. Store original uploaded or Drive-imported file bytes in a Cloudflare R2 bucket bound to the Site as ``BUCKET``.
5. Do not use browser local storage as the source of truth. Data must be available from any device after I sign into the same ChatGPT account and open the published Site.
6. The Site itself must not attempt to browse Google Drive from client-side code. The ChatGPT Work automation is the secure bridge: it reads Drive through the authorized connector and posts imports to the Site's protected server endpoint.
7. Create or reuse one dedicated Google Drive folder exactly once. Use the exact folder name ``Ledgerly Financial Inbox``. If a folder with that exact name already exists in my Drive, reuse it instead of creating another one.
8. Schedule the Drive import exactly once per day at 8:00 AM in my local timezone. Do not create duplicate schedules. If my timezone is unavailable, ask one concise question for it before creating the schedule.
9. On first launch, all financial datasets must be empty. Do not insert demo/sample transactions, balances, budgets, goals, subscriptions, recurring bills, rules, tags, receipts, statements, invoices, or documents.
10. Starter category names and account-picker names are configuration definitions only, not financial records. They must not contain balances or create transactions.
11. Every visible button, menu, tab, icon, filter, form, and modal described below must work. No placeholder controls and no broken icon glyphs.
12. Use ``lucide-react`` icons or another installed vector icon library. Do not use unsupported font-icon characters.
13. Make all body and helper text readable. Normal body text should be approximately 14–16 px; helper text and table metadata should not be smaller than 12 px. Form controls and touch targets should be at least 44 px high on mobile where practical.
14. Never expose access tokens, account numbers, financial document contents, or secrets in logs, source code, notifications, or chat responses.

2. Build and setup sequence

1. Execute the work in this order so the integrations point to the correct resources:
1. Verify that the Sites and Google Drive connections are available using harmless read-only checks. If either connector asks me to Connect or Reconnect, stop only for that authorization and continue immediately afterward.
2. Search my Drive for a folder whose name is exactly ``Ledgerly Financial Inbox``.
3. If exactly one matching folder exists, reuse it. If none exists, create it once. If multiple exact matches exist, show me the matches and ask which one to use; do not create another folder.
4. Record the selected folder's ID, name, and URL for later use. Do not expose its ID as a secret; it is configuration, but do not hard-code someone else's folder ID.
5. Build the Site, D1 schema, R2 binding, pages, APIs, and empty states in this prompt.
6. Publish one private checkpoint deployment and capture its exact URL/slug.
7. Confirm the Site's ``/api/drive-sync`` endpoint is live before creating the schedule.
8. Create one daily 8:00 AM automation using the exact Site URL and exact Drive folder selected in this setup. If an automation already exists for the same Site and folder, update/reuse it instead of creating a duplicate.
9. Perform one safe manual test using an empty folder listing or a tiny non-sensitive test file only if needed. Do not create sample financial records for the test. Remove any temporary test artifact you created.
10. Verify the published Site on desktop and mobile widths, then report the Site link, Drive folder link, schedule time/timezone, storage configuration, and test results.

3. Empty-start contract

The first real user session must start with these values: - ``transactions = []`` - ``documents = []`` - ``goals = []`` - ``budgets = []`` - ``subscriptions = []`` - ``recurring = []`` - ``rules = []`` - ``tags = []`` - ``dismissedPatterns = []`` - ``selectedPeriod = "all-time"`` - assets total = ``0``, liabilities total = ``0``, but the Net Worth card must display ****Not set**** until the user explicitly saves asset or liability totals - all charts display polished empty states instead of invented data - totals derived from transactions display ``$0.00``

- no fake "recent activity," "upcoming payment," insights, or trend percentages You may create these starter category definitions because they are lookup configuration, not financial data: `Housing, Groceries, Shopping, Dining, Transportation, Utilities, Subscriptions, Insurance, Health, Entertainment, Income, Needs review, Other` You may create these starter account names as lookup configuration only, with no balances: `Main Checking, Everyday Visa, Rewards Card, Cash` The user must be able to remove any of those definitions in Settings. If you choose to initialize the account list as empty instead, disable the account selector with a clear ****Add an account in Settings**** action until an account exists. Never create balances automatically. **## 4.**

Technical architecture and durable storage Use a React/Vinext-compatible Sites application with server routes. Configure `.openai/hosting.json` with these logical bindings: - D1 database binding: ``DB`` - R2 object bucket binding: ``BUCKET`` Use D1 for structured data and metadata. Use R2 only for original uploaded/imported file bytes. The browser should fetch state from server APIs and update the server after every edit. **### 4.1 D1 tables** Create the schema idempotently so deployments and requests do not destroy existing data. **##### `transactions`** - ``id`` TEXT PRIMARY KEY - ``date`` TEXT NOT NULL, ISO ``YYYY-MM-DD`` - ``merchant`` TEXT NOT NULL - ``category`` TEXT NOT NULL DEFAULT ``Needs review`` - ``amount`` REAL NOT NULL and stored as a positive magnitude - ``type`` TEXT NOT NULL with allowed values ``expense`` or ``income`` - ``account`` TEXT NOT NULL DEFAULT ``Imported account`` - ``tags`` TEXT NOT NULL DEFAULT ``[]``, stored as a JSON array - ``receipt`` INTEGER NOT NULL DEFAULT ``0`` - ``source`` TEXT NOT NULL, such as ``manual``, ``csv``, ``document``, or ``google-drive`` - ``fingerprint`` TEXT NOT NULL UNIQUE - ``createdAt`` TEXT NOT NULL, ISO timestamp **##### `tags`** - ``name`` TEXT PRIMARY KEY - ``createdAt`` TEXT NOT NULL **##### `rules`** - ``id`` TEXT PRIMARY KEY - ``whenText`` TEXT NOT NULL - ``thenText`` TEXT NOT NULL - ``enabled`` INTEGER NOT NULL DEFAULT ``1`` - ``createdAt`` TEXT NOT NULL **##### `settings`** - ``key`` TEXT PRIMARY KEY - ``value`` TEXT NOT NULL, JSON or scalar text as appropriate - ``updatedAt`` TEXT NOT NULL Use settings keys for categories, accounts, goals, budgets, subscriptions, recurring payments, dismissed recurring-pattern keys, assets total, liabilities total, whether net worth was explicitly configured, the persisted global date period ``selectedPeriod``, Drive folder metadata, Drive sync metadata, processed Drive file IDs, ``driveResetAt``, and ``freshStart``. **##### `documents`** - ``id`` TEXT PRIMARY KEY - ``filename`` TEXT NOT NULL - ``mimeType`` TEXT NOT NULL - ``size`` INTEGER NOT NULL - ``objectKey`` TEXT NOT NULL UNIQUE - ``status`` TEXT NOT NULL with values such as ``queued``, ``stored``, or ``review`` - ``source`` TEXT NOT NULL, such as ``upload`` or ``google-drive`` - ``createdAt`` TEXT NOT NULL **### 4.2 R2 object storage** - Store original receipt, invoice, statement, spreadsheet, image, PDF, and other supported document bytes. - Enforce a maximum file size of 20 MB per file. - For manual uploads, use a safe object key such as ``uploads/-``. - For Drive imports, use ``drive-inbox/-``. - Add Drive file ID and modified time as object metadata where supported. - Do not expose raw bucket URLs publicly. **### 4.3 State API** Implement ``GET /api/state`` to return one normalized JSON payload containing: - up to 5,000 transactions, newest first - all tags - all rules - decoded settings - up to 100 document metadata rows, newest first Never return original file bytes through the state endpoint. **### 4.4 Transaction API** Implement ``/api/transactions``: - ``POST`` accepts one transaction or a batch. - Validate a non-empty merchant/source, a valid date, a positive finite amount, and ``type`` of ``expense`` or ``income``. - Normalize tags by trimming, removing blanks, and deduplicating case-insensitively. - Build this duplicate fingerprint before insertion: ``date + "|" + merchant.trim().toLowerCase() + "|" + amount.toFixed(2) + "|" + account.trim().toLowerCase()`` - Rely on the unique fingerprint plus a pre-check to prevent duplicates. - Return inserted and duplicate counts clearly for a batch. - ``PATCH`` updates a transaction's category and/or tags by ID and returns the saved row. - ``DELETE`` deletes a transaction by exact ID. Apply enabled categorization rules only after duplicate detection. Rules must not cause a duplicate to be inserted. **### 4.5 Preferences API** Implement ``PUT /api/preferences`` to save managed categories, accounts, tags, rules, goals, budgets, subscriptions, recurring entries, dismissed pattern keys, asset/liability totals, and other settings. Validate and normalize all arrays. Never reset unrelated settings when one preference group is updated. **### 4.6 Document API** Implement ``POST /api/documents`` as multipart upload: - accept one or multiple supported files - reject files over 20 MB with a readable error - write original bytes to R2 - write document metadata to D1 - use status ``queued`` until parsed, ``stored`` after successful storage/processing, and ``review`` when extraction is uncertain - never create a financial transaction from uncertain

values **###** 4.7 Complete data wipe API Implement ``DELETE /api/state``. It must require the JSON confirmation value exactly: ``DELETE ALL LEDGERLY DATA`` On a valid request: 1. Delete all transactions, documents, rules, tags, and settings rows from D1. 2. Delete every object belonging to this Ledgerly Site from R2. 3. Recreate only the structural empty-state settings needed for the app. 4. Save ``freshStart = true``. 5. Save ``driveResetAt`` as the current ISO timestamp. 6. Save assets and liabilities as zero and ``netWorthConfigured = false``. 7. Reset ``selectedPeriod = "all-time"`` so the freshly wiped Site opens in ****All time****. 8. Return a clear success payload. The wipe does not delete files from Google Drive. The daily automation must read ``driveResetAt`` and never reimport a Drive file whose modified time is at or before that timestamp. This lets the user start fresh without old inbox files repopulating the Site. **##** 5. Global application shell and visual system Build a calm, polished financial interface: - light gray application background - white cards with subtle borders and soft shadows - primary violet accent around ``#6558D3`` - green for positive income/savings states - orange for spending or caution states - blue for secondary savings/information states - dark navy summary panels where contrast is appropriate - rounded cards around 14–16 px radius - desktop left sidebar approximately 238 px wide - desktop sticky top bar approximately 76 px high - generous whitespace and consistent alignment Desktop navigation appears in the sidebar. Mobile uses a compact top bar and a horizontally scrollable bottom navigation that can reach every tab. Do not hide later tabs off-screen without scrolling affordance. The navigation order must be: 1. Dashboard 2. Transactions 3. Recurring 4. Subscriptions 5. Budgets 6. Goals 7. Documents 8. Rules 9. Settings The global top bar must include these common actions: - ****Drive sync**** - ****Import**** - ****Add entry**** All dialogs must trap focus, close with a visible close button and Escape, have labels/ARIA attributes, and remain usable on narrow phones. Show inline success/error feedback and disable submit buttons while saving. **##** 6. Dashboard page **###** 6.1 Date period Provide a working period selector with: - All time - This month - Last month - Last 3 months - Last 6 months - This year Use ****All time**** on the first-ever visit and whenever no valid saved period exists. ``All time`` means there is no start-date cutoff: include every saved transaction through the present. Persist the global period under the D1 settings key ``selectedPeriod``. Allowed values are ``all-time``, ``this-month``, ``last-month``, ``last-3-months``, ``last-6-months``, and ``this-year``. When the user selects a period: 1. Recalculate all affected views immediately. 2. Save the new value through ``PUT /api/preferences`` without resetting any other preference. 3. Keep that value after refresh, sign-out/sign-in, opening the Site on another device, or starting a later session. 4. On every launch, load the saved value from ``GET /api/state`` before rendering date-dependent totals. Avoid briefly showing another period while state loads. 5. If saving fails, restore the previously saved selection and show a clear error. The selected period must actually recalculate date-dependent totals, charts, recent activity, and transaction views. Do not make it a decorative button. The Dashboard and Transactions pages must share the same persisted selection, so changing it on either page updates the other. **###** 6.2 Summary cards Show four cards across on wide desktop and responsively stack them on smaller screens: 1. ****Net Worth**** - Formula: ``total assets - total liabilities``. - Assets and liabilities are user-entered in Settings; transaction cash flow does not automatically become net worth. - Until the user explicitly saves totals, display ``Not set``, explanatory text, and a link/button to Settings. - After setup, show the calculated currency value. 2. ****Income**** - Sum all ``income`` transactions in the selected period. 3. ****Spending**** - Sum all ``expense`` transactions in the selected period. 4. ****Savings rate**** - Formula: ``((income - spending) / income) * 100``. - If income is zero, show ``0%`` and avoid division errors. The small areas at the bottom of these cards must not be unexplained decorative boxes. Use a labeled calculation strip or a real compact trend derived from saved data. If there is not enough history, show a readable ``No trend yet`` state. Do not invent month-over-month arrows or percentages. Show a comparison only when both current and prior-period real data exist. **###** 6.3 Dashboard content - ****Cash flow chart****: a responsive line/area chart using saved dated transactions. Provide up to seven monthly points. Income is violet/green and expenses are neutral gray/orange. If empty, show ``Import or add transactions to see cash flow.`` - ****Spending by category****: donut/pie chart grouped from real expense transactions for the selected period, with labels, accessible legend, values, and percentages. If empty, show an empty state. - ****Recent activity****: five newest transactions in the selected period with merchant, date, category, account, and signed amount. - ****Ledgerly insight****: show useful, factual insights only, such as a count of transactions in ``Needs``

review`. Do not generate fake savings advice from absent data. - ****Coming up:**** show confirmed recurring expenses/subscriptions due soon. If none, say so and link to Recurring. ##

7. Transactions page Provide a responsive transaction table/list with: - search by merchant, category, or tag - account filter - category filter - shared working date-period selector - columns/fields: date and merchant, category, account, tags, amount - income formatted with a leading plus and positive color - expenses formatted with a leading minus and standard/dark color - receipt-matched indicator when applicable ### 7.1 Inline category editing The category cell for every transaction must be directly editable. Tapping/clicking it opens a dropdown of the current managed categories. Save immediately to D1 through `PATCH /api/transactions`, update the row without a full reload, and show an error if persistence fails. ### 7.2 Inline tag editing - Render tags as removable pills. - Tapping a pill removes that tag after saving. - Place a visible ****+**** control at the end of each row's tags. - The ****+**** opens a tag-only modal. It must not ask the user to create or select a category. - The modal lets the user select one or more existing tags and optionally create one simple new tag by typing only the tag name. - Creating a tag must not require a category, rule, color, or other metadata. - Save both the global tag definition and the transaction's updated tag array. ### 7.3 Add entry The global ****Add entry**** button opens a modal with: - segmented Expense / Income selection - amount, initially blank - merchant or source, initially blank - date, defaulted to today but editable - category selector from managed categories - account selector from managed accounts - tags selector with simple add-new-tag support - checkbox `I have a receipt to attach` - optional file picker shown when the receipt box is enabled The form must never prefill a fake amount or merchant. Validate before save, persist the transaction, optionally persist the receipt, close on success, and refresh all affected summaries. ##

8. Imports and duplicate handling ### 8.1 CSV statement import The global ****Import**** button must support CSV bank or card statements. 1. Let the user choose a CSV and preview the detected columns. 2. Recognize common headers for date, description/merchant, amount, debit, credit, category, and account. 3. If mapping is ambiguous, show a mapping step; do not guess silently. 4. Normalize dates to `YYYY-MM-DD`. 5. Store the amount as a positive magnitude and map debit/negative rows to `expense`, credit/positive rows to `income`, honoring the statement's actual sign convention. 6. Preserve a supported statement category; otherwise use `Needs review`. 7. Save `source = csv` and `receipt = false`. 8. Run the same fingerprint duplicate detector as every other path. 9. Present an import result containing inserted, duplicate, skipped, and needs-review counts. 10. Never create placeholder rows for unparseable lines. ### 8.2 Manual document import The Import flow and Documents page must also accept receipts, invoices, images, PDFs, spreadsheets, and other supported documents: - store the original file in R2 first - extract merchant/payee, date, total, and type only when grounded in the document - categorize only when supported; otherwise use `Needs review` - set `receipt = true` for a receipt/invoice-backed transaction - flag uncertain extractions as `review` instead of inventing values - use the same duplicate fingerprint before inserting any extracted transaction ### 8.3 Duplicate rules Duplicate detection must be centralized and apply to manual entry, CSV, documents, and Drive. Use the transaction fingerprint plus the unique database constraint. A duplicate must not be inserted twice even if two imports happen simultaneously. Keep the original document metadata when appropriate, but clearly report that its transaction was a duplicate. ##

9. Automatic recurring and subscription detection Automatically detect recurring payments from real expense transactions; do not call any transaction recurring based on its name alone. ### 9.1 Merchant normalization For pattern matching, lowercase and trim merchant names, remove punctuation, remove terminal `#` plus digits, collapse whitespace, and remove long reference-number digit sequences. Keep the original display merchant on the transaction. ### 9.2 Candidate grouping and cadence Group expense transactions by normalized merchant. Require at least two unique transaction dates. Calculate consecutive-day intervals and classify only within these windows: - weekly: 5–9 days - biweekly: 12–17 days - monthly: 24–40 days - quarterly: 75–110 days - annual: 330–400 days Reject a candidate when the dominant interval does not fit one of those windows. Use average amount variation limits: - subscription candidate: no more than 20% - other recurring candidate: no more than 35% ### 9.3 Subscription hints Treat a stable pattern as a subscription candidate when its category/tag includes `subscription`, or its normalized merchant contains a strong hint such as: `netflix`, `spotify`, `hulu`, `disney`, `youtube`, `icloud`, `dropbox`, `adobe`, `microsoft`, `amazon prime`, `patreon`, `membership`, `studio`, `gym`, `openai`, `chatgpt`, `canva`, `notion`, `zoom`, `slack`, `github` ### 9.4

Recurring-bill hints Treat a stable pattern as another recurring-payment candidate when its normalized merchant/category/tags contain a strong hint such as: `mortgage, rent, loan, insurance, utility, utilities, electric, water, internet, phone, mobile, daycare, tuition, lease, car payment, auto payment, hoa, property tax` #### 9.5 Protection against false positives A merchant with no strong hint can be suggested only after at least three stable monthly, quarterly, or annual occurrences with no more than 3% amount variation. Do not auto-suggest routine weekly shopping or grocery spending merely because it repeats. #### 9.6 Confidence and next date - ****High confidence:**** at least three occurrences, amount variation at most 12%, and interval jitter at most five days. - ****Likely:**** meets the base pattern but not all high-confidence thresholds. - Calculate the next occurrence calendar-aware, preserving day-of-month behavior where possible. - Convert to monthly equivalents using: weekly `amount \* 52 / 12`; biweekly `amount \* 26 / 12`; monthly `amount`; quarterly `amount / 3`; annual `amount / 12`. #### 9.7 User control Suggestions must show merchant/service, category, cadence, occurrence count, confidence, average charge, estimated monthly equivalent, and next expected date. - ****Keep**** saves it as a confirmed recurring payment or subscription. - ****Ignore**** persists a stable dismissed-pattern key so the suggestion remains hidden across devices and future sessions. - Settings includes ****Restore ignored suggestions****. - Detection never silently creates a confirmed item. The user must choose Keep or manually add one. ## 10. Recurring page Provide: - an `Active detection` status banner - a suggestions panel driven by the algorithm above - combined estimated monthly and annual commitment for confirmed entries and visible detected suggestions, without double counting - next expected payment - confirmed recurring-payment list - manual ****Add recurring payment**** action - edit/manage action for each confirmed item A recurring entry contains name, category, amount, cadence, next date, optional account, and active status. No sample recurring entries may be created. ## 11. Subscriptions page Provide: - automatic subscription suggestions from the same detection engine - estimated monthly and annual subscription totals - next renewal - confirmed subscription list - manual ****Add subscription**** action - edit/manage action for each confirmed subscription A subscription contains service name, group/category, amount, cadence, next renewal date, optional account, and active status. No sample subscriptions may be created. ## 12. Budgets page Start empty and show a clear ****Create budget**** action. Each budget has category, monthly limit, and active status. For each saved budget, calculate actual spending from transactions in the selected month and show: - category - spent amount - budget limit - remaining amount - progress bar and percentage - clear over-budget state Include a budget-health summary/ring calculated from real saved budgets. Provide a working ****Adjust budgets**** modal for editing or deleting existing budgets. Every save persists through the preferences API. ## 13. Goals page Start empty and show a clear ****Create goal**** action. A goal has name, target amount, current saved amount, optional due date, and optional note. Goal cards show: - name and due date - current amount and target amount - remaining amount - progress bar/percentage - edit and delete actions The ****Add goal**** button and edit controls must work and persist. Do not create example goals. ## 14. Documents page Provide two primary cards: 1. ****Upload documents**** for multiple receipts, statements, invoices, PDFs, images, CSVs, and supported spreadsheets, maximum 20 MB each. 2. ****Google Drive inbox**** showing the dedicated folder name/link, last sync timestamp, active schedule badge, last result counts, and an action to view the folder or run a sync flow when available. Below them, show a secure document vault list with filename, type, size, source, status, and imported date. Clearly distinguish `stored`, `queued`, and `review`. Empty state: `No documents yet. Upload a file or add one to your Drive inbox.` Do not claim files are encrypted beyond the actual platform's storage protections. Do not display raw R2 object keys to the user. ## 15. Rules and tags page #### 15.1 Categorization rules Show saved rules as `When ... then ...` rows with an enabled toggle, edit, and delete actions. A ****Create rule**** modal accepts: - `whenText`: a plain-language merchant/source condition - `thenText`: the target category and/or tag action - enabled state Start with no rules. Do not seed merchant-specific examples. Apply enabled rules to future imports after duplicate detection. Do not rewrite historical transactions unless the user explicitly requests a separate bulk action. #### 15.2 Tag management Show all tags with usage counts. ****Create tag**** asks for one tag name only. It must not ask for a category. Allow deletion with confirmation. Removing a tag definition should remove it from future selectors; ask before stripping it from historical transactions. ## 16. Settings page #### 16.1 Net worth setup Provide editable totals for: - total assets - total liabilities Show a live preview using `assets - liabilities`.

Save the totals and `netWorthConfigured = true`. Explain clearly that Net Worth is not calculated from monthly income minus expenses; it is the user's assets minus liabilities. #### 16.2 Managed categories, tags, and accounts Provide separate sections for categories, tags, and accounts. In each section the user can add a simple name and remove an item with confirmation. - Adding a tag requires only its name. - Categories and accounts are independent from tags. - Removing a category/account removes it from future pickers but keeps the existing historical label on prior transactions. - Prevent exact duplicates after case-insensitive trimming. - Prevent deletion of a category/account currently required by an open form; show a useful message instead of crashing. #### 16.3 Automatic detection settings Explain the recurring/subscription detection behavior in plain language. Show the number of ignored suggestions and a working **Restore ignored suggestions** button. #### 16.4 Google Drive sync settings Show the folder name/link, schedule, timezone, last sync, status, imported count, duplicate count, review count, and errors. Do not show bearer tokens. Provide clear instructions: add a receipt, CSV, statement, invoice, or supported document to the dedicated folder and it will be checked at 8:00 AM daily. #### 16.5 Danger zone Provide **Erase all Ledgerly data**. The confirmation modal must: 1. explain that Site database records and stored R2 file copies will be deleted 2. explain that original Google Drive files will remain 3. require the user to type `DELETE` 4. enable the destructive button only after an exact match 5. call `DELETE /api/state` with the server confirmation `DELETE ALL LEDGERLY DATA` 6. clear the in-memory UI after success and return to polished empty states After wiping, the 8:00 AM automation remains configured but must not reimport Drive files modified at or before `driveResetAt`. ## 17. Google Drive sync endpoint Implement `/api/drive-sync` as a protected server endpoint. #### 17.1 `GET /api/drive-sync` Return: - dedicated folder metadata - schedule metadata `{ time: "08:00", timezone: "", cadence: "daily" }` - last sync timestamp and status - last imported, duplicate, stored-file, review, and error counts - `processedFileIds`, capped to the most recent 5,000 IDs - `resetAt`, sourced from `driveResetAt` #### 17.2 `POST /api/drive-sync` Accept a batch payload containing grounded transactions, file objects, and concise errors. For each Drive transaction: - normalize date, merchant, amount, type, account, category, tags, and receipt - use `source = google-drive` - use account `Drive import` only if the document has no grounded account and no better mapped account - add the tag `Drive import` - use the same duplicate fingerprint as all other imports For each file: - accept Drive file ID, filename, MIME type, modified time, base64 original content when available, and status - enforce the 20 MB limit after decoding - store bytes under `drive-inbox/-` - store D1 metadata with source `google-drive` - mark uncertain content as `review` Mark a Drive file ID processed only after its accepted data and/or original bytes have been successfully stored. If transfer or processing fails, leave it unprocessed so the next daily run retries it. Return: - overall status: `complete` or `partial` - `lastSyncedAt` - transactions imported - duplicates skipped - files stored - files needing review - concise errors Protect this endpoint with the Site's owner access plus the temporary Sites/ChatGPT Work authorization mechanism used by the automation. Never place a temporary bearer token in the repository, D1, R2, browser bundle, Drive file, automation description, or chat output. ## 18. Create the daily 8:00 AM automation After the Site and folder exist, create one automation titled **Sync Ledgerly Inbox**. Use an exact daily schedule, not an approximate background cadence. Create the equivalent of: ```text BEGIN:VEVENT DTSTART: RRULE:FREQ=DAILY END:VEVENT ``` Set `timing_mode` to exact schedule and set the default timezone to my actual timezone. Use the following run instructions, replacing placeholders with the exact Site and Drive folder created for me: #### Automation run instructions You maintain the private Ledgerly financial Site at `` using the dedicated Google Drive folder `` with ID ``. 1. Verify Google Drive and Sites access with harmless reads. Resolve the exact existing Site. Obtain a temporary Site authorization/bypass bearer for this run through the Sites connection. Never display, persist, log, or place the token in source code. 2. Call `GET /api/drive-sync` with `OAI-Sites-Authorization: Bearer ``. Treat `processedFileIds` as the authoritative Drive duplicate ledger. Read `resetAt`; ignore every Drive file whose modified time is at or before `resetAt`. 3. List only direct children of the dedicated Drive folder. Process only file IDs that are absent from `processedFileIds` and whose modified time is after `resetAt`. Do not move, rename, edit, share, trash, or delete Drive files. 4. For CSV statements, parse real rows into date, merchant, positive magnitude amount, income/expense type, account when grounded, category only when supported, tags, `receipt=false`, and `source=google-drive`. Preserve the statement's actual debit/credit meaning. Never invent

missing values. 5. For receipts, invoices, PDFs, images, spreadsheets, and supported documents, extract merchant/payee, date, total, type, and category only when grounded. Set `receipt=true` for receipt/invoice-backed transactions. When material fields are uncertain, set file status to `review` and do not invent a transaction. 6. Download original file bytes when available. Post each new file or a small safe batch to `/api/drive-sync` with Drive file ID, filename, MIME type, modified time, base64 content, status, grounded transactions, and the temporary authorization header. The Site endpoint owns durable storage and duplicate detection. 7. If a file cannot be read or transferred, include a concise error and leave its file ID unprocessed so a later run retries it. 8. Verify the Site response. Notify me with counts for new transactions, stored files, duplicates, and items needing review. If there were no new files, say the inbox was checked and is up to date. Include the Site link and any concise failures. Never expose financial document contents, full account numbers, tokens, or secrets. Never change Site access, Drive sharing, or external data beyond importing it into the private Site. ## 19. Responsive behavior Test at minimum at approximately 1440 px desktop, 768 px tablet, and 390 px mobile widths. - Summary cards reflow from four columns to two and then one. - Charts resize without clipping labels. - Transaction rows become a readable stacked layout or horizontally scroll within a clearly contained table. - Inline category and tag controls remain tappable. - Modals fit within the viewport, scroll internally when needed, and keep actions reachable. - Mobile bottom navigation reaches all nine tabs. - No text overlaps, tiny labels, cut-off amounts, or horizontal page overflow. ## 20. Reliability and security behavior - Show loading, empty, success, and failure states on every data surface. - Use server-side validation for all writes; never trust client input alone. - Use parameterized D1 queries. - Escape or safely render all merchant names, tags, filenames, and rule text. - Use random UUIDs for record IDs. - Make schema creation and preference initialization idempotent. - Avoid race conditions by enforcing unique transaction fingerprints in D1. - Do not log full imported files, account numbers, bearer tokens, or transaction payloads. - If D1 or R2 is unavailable, show a clear error and do not pretend the save succeeded. - Keep prior successful data intact when one file in a batch fails; report partial status. ## 21. Required functional verification Before declaring completion, verify all of the following against the published Site, not only the local preview: 1. First load contains no sample financial data. 2. Dashboard empty states render correctly, Net Worth says `Not set`, and the first-ever period is `All time`. 3. Add one temporary manual transaction, refresh the page, and confirm persistence from D1; then delete that test transaction so the delivered Site is empty. 4. Add/remove a temporary tag without creating a category; remove the tag afterward. 5. Inline category and tag editing persists after refresh. 6. CSV import parses a controlled temporary file, reports duplicates correctly on a second import, and all temporary imported records/files are deleted before handoff. 7. Assets and liabilities calculate Net Worth as assets minus liabilities; restore the delivered state to not configured afterward. 8. Create/edit/delete temporary budget, goal, subscription, recurring item, rule, category, and account entries; remove all temporary data afterward. 9. Document upload stores bytes in R2 and metadata in D1; remove the temporary object/metadata afterward. 10. Data wipe requires the exact confirmation flow and sets `driveResetAt`. Do not use the final destructive wipe if it would interfere with the automation test; test it before final empty initialization or in an isolated preview database. 11. The Drive folder exists exactly once and the automation references its exact ID. 12. The automation is scheduled once at 8:00 AM in the correct timezone. 13. Every top action, navigation tab, modal, period option, filter, add/edit/delete action, Drive link, and Settings control works. 14. No icon is missing and no important text is smaller than the readability rules. 15. Desktop, tablet, and mobile layouts have no clipping or inaccessible controls. 16. Refreshing and opening the Site from another signed-in browser session returns the same D1-backed state. 17. Changing the period to `This month`, refreshing, and opening a new session restores `This month`; changing it back to `All time` also persists. Leave the delivered Site set to `All time`. After testing, remove every temporary transaction, tag, rule, budget, goal, recurring item, subscription, document, asset/liability value, and test file created in the Site. Leave the published Site in the exact empty-start state. Do not delete the dedicated Drive folder or the daily automation. ## 22. Completion response When finished, give me a concise handoff containing: - the published private Site link - the dedicated Drive folder link - confirmation that the Site starts with no sample financial data - D1 and R2 storage summary - the exact automation schedule and timezone - the next scheduled run - a short explanation of how duplicate detection works - a short explanation of

how recurring/subscription detection works - confirmation that every tab and button was tested on desktop and mobile - any one-time authorization step that still requires my action Do not call the project complete if a visible control is still a placeholder, if the Drive folder or automation was not created, if data is only stored in the browser, or if the delivered Site contains sample financial records. # MASTER PROMPT — END